

```
>>> numberOfPets['cats']
10
```

But because this pattern is also common, dictionaries have a more pythonic `setdefault()` method. The following code is equivalent to the previous example:

```
>>> # Pythonic Example
>>> numberOfPets = {'dogs': 2}
>>> numberOfPets.setdefault('cats', 0) # Does nothing if 'cats' exists.
0
>>> numberOfPets['cats'] += 10
>>> numberOfPets['cats']
10
```

If you're writing `if` statements that check whether a key exists in a dictionary and sets a default value if the key is absent, use the `setdefault()` method instead.

Use `collections.defaultdict` for Default Values

You can use the `collections.defaultdict` class to eliminate `KeyError` errors entirely. This class lets you create a default dictionary by importing the `collections` module and calling `collections.defaultdict()`, passing it a data type to use for a default value. For example, by passing `int` to `collections.defaultdict()`, you can make a dictionary-like object that uses `0` for a default value of nonexistent keys. Enter the following into the interactive shell:

```
>>> import collections
>>> scores = collections.defaultdict(int)
>>> scores
defaultdict(<class 'int'>, {})
>>> scores['Al'] += 1 # No need to set a value for the 'Al' key first.
>>> scores
defaultdict(<class 'int'>, {'Al': 1})
>>> scores['Zophie'] # No need to set a value for the 'Zophie' key first.
0
>>> scores['Zophie'] += 40
>>> scores
defaultdict(<class 'int'>, {'Al': 1, 'Zophie': 40})
```
